

Unión Europea

Fondo Social Europeo

*El FSE invierte en tu futuro*

# Laravel

Formularios

Pepe Devesa



GENERALITAT VALENCIANA

Conselleria d'Educació, Investigació, Cultura i Esport



GOBIERNO DE ESPAÑA

MINISTERIO DE EDUCACIÓN, CULTURA Y DEPORTE

# Continguts

**1 Definición y uso de formularios** **3**

**2 Creación y envío de formularios** **3**

2.1 Actualizaciones y borrados . . . . . 5

# 1 Definición y uso de formularios

El envío de formularios en Laravel implica, por un lado, definir un formulario, empleando HTML sencillo junto con algunas opciones ofrecidas por Blade. Por otra parte, debemos recoger los datos enviados por el formulario en algún método de algún controlador, y procesarlos adecuadamente.

# 2 Creación y envío de formularios

Si definimos un formulario en una vista, se define con los conceptos que ya sabemos de HTML. Como único añadido, en el campo `action` del formulario podemos utilizar Blade y la función route para indicar el nombre de ruta a la que queremos enviar el formulario. Veamos, por ejemplo, cómo definir un formulario para dar de alta nuevos libros en nuestro proyecto de biblioteca. Creamos una vista llamada `create.blade.php` en la subcarpeta `resources/views/libros`, con un contenido como éste:

```
<h1>Nuevo libro</h1>
<form  action="{{ route('libros.store') }}" method="POST">
<div class="row mb-3">
    <div class="form-group">
        <label for="titulo">Título:</label>
        <input type="text" class="form-control" name="titulo"id="titulo">
    </div>

    <div class="form-group">
        <label for="editorial">Editorial:</label>
        <input type="text" class="form-control" name="editorial"id="
           editorial">
    </div>

    <div class="form-group">
        <label for="precio">Precio:</label>
        <input type="text" class="form-control" name="precio"id="precio">
    </div>

    <div class="form-group">
        <label for="autor">Autor:</label>
        <select class="form-control" name="autor" id="autor">
        @foreach ($autores as $autor)
            <option value="{{ $autor->id }}">{{ $autor->nombre }}</option>
        @endforeach
        </select>
    </div>

    <input type="submit" name="enviar" value="Enviar" class="btn btn-dark
       btn-block">
```

```
</div>
</form>
```



Un segundo añadido más que tenemos que tener en cuenta es que Laravel por defecto protege de ataques XSS (Cross Site Scripting) de suplantación de identidad, por lo que obtendremos un `error de tipo 419` si enviamos un formulario no validado. Para solucionar este problema, basta con utilizar la directiva `@csrf` en el formulario, que añade un campo oculto con un token de validación:

```
<h1>Nuevo libro</h1>
<form  action="{{ route('libros.store') }}" method="POST">
    @csrf

    ...
```

En el método create:

```
  /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        //
        $autores = Autor::get(); //para cargar los autores en el
            desplegable
        return view('libros.create',compact('autores'));
    }
```

Por otra parte, el método `store` se encargará de recoger los datos de la petición a través del parámetro `Request` de dicho método. Disponemos de un método `get` para acceder a cada campo del formulario a partir de su nombre:

```
public function store(Request $request)
{
    $libro = new Libro();
    $libro->titulo = $request->get('titulo');
    $libro->editorial = $request->get('editorial');
    $libro->precio = $request->get('precio');
    $libro->save();

    return redirect()->route('libros.index'); //Redireccionar al listado.
}
```

Podemos emplear también algún método auxiliar de la petición, como `has`, que comprueba si existe un campo con un nombre determinado:

```
public function store(Request $request)
{
    if($request->has('titulo'))
    {
        ...
    }
}
```

### 2.1 Actualizaciones y borrados

Por defecto, el atributo method de un formulario sólo admite los valores `GET` o `POST`. Si queremos enviar un formulario de actualización o borrado, éste debe ir asociado a los métodos `PUT` o `DELETE`, respectivamente. Para esto, podemos emplear dentro del mismo formulario la directiva `@method` , indicando el nombre del método que queremos usar:

```
<form ...>
    @csrf
    @method('PUT')
    ...
</form>
```

Por ejemplo, para borrar libros en nuestra aplicación de biblioteca, podríamos añadir un formulario como este en la ficha del libro (vista `resources/views/libros/show.blade.php`):

```
<form action="{{ route('libros.destroy', $libro->id) }}" method="POST">
    @csrf
    @method('DELETE')
    <input type="submit" class="btn btn-danger" value="Borrar libro" />
</form>
```

Observa cómo le pasamos a la ruta el id del libro a borrar, para que le llegue como parámetro al método `destroy` del controlador. Dentro de este método, buscamos el libro afectado, lo borramos, y mostramos el listado de libros nuevamente:

```
public function destroy($id)
{
    $libro = Libro::findOrFail($id);
    $libro->delete();

    return redirect()->route('libros.index');
}
```